public static void sleep(long millisecond) throws InterruptedException
----

sleep() is a predefined static method of Thread class which accepts long ms as a parameter. [parameter must be a positive value]

The main purpose of sleep method to put a thread into temporarly waiting state OR TIMED_WAITING State, The waiting period of the thread will depend upon the time specified by the user as a parameter of sleep() method.

Example :
Thread.sleep(1000); //Thread will wait here for 1 second.

It is throwing a checked Exception i.e InterruptedException because there may be chance at runtime, this sleeping thread may be interrupted by a thread so provide either try-catch or declare the method as throws.

//Program :

```
package com.ravi.sleep;

class Test extends Thread
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();

        for(int i=1; i<=10; i++)
        {
            IO.println(i+" by "+name+" thread");
          try
          {
                Thread.sleep(1000);
          }
          catch(InterruptedException e)
          {
                IO.println("Thread is interrupted "+e);
          }
        }
    }
}
public class SleepDemo1
{
    public static void main(String[] args)
    {
       Test t1 = new Test();
       t1.setName("Child1");
       t1.start();

    }
}
```

```
package com.ravi.sleep;

class Foo extends Thread
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();  //Thread1

        for(int i=1; i<=5; i++)
        {
            IO.println(i+" by "+name+" thread");
            try
            {
                Thread.sleep(1000, 2000);
            }
            catch(InterruptedException e)
            {

            }
        }
    }
}

public class SleepDemo2
{
    public static void main(String[] args)
    {
        Foo f1 = new Foo();
        Foo f2 = new Foo();

        f1.setName("Thread1");
        f2.setName("Thread2");

        f1.start();
        f2.start();

    }

}
```

Old Thread life Cycle :
----

* Before JDK 1.5V

* As we know a thread is well known for Independent Execution but still It has life cycle.

* During the life cycle, It can pass through only one phase OR one state at a time.

* The following are the phases OR states of a thread

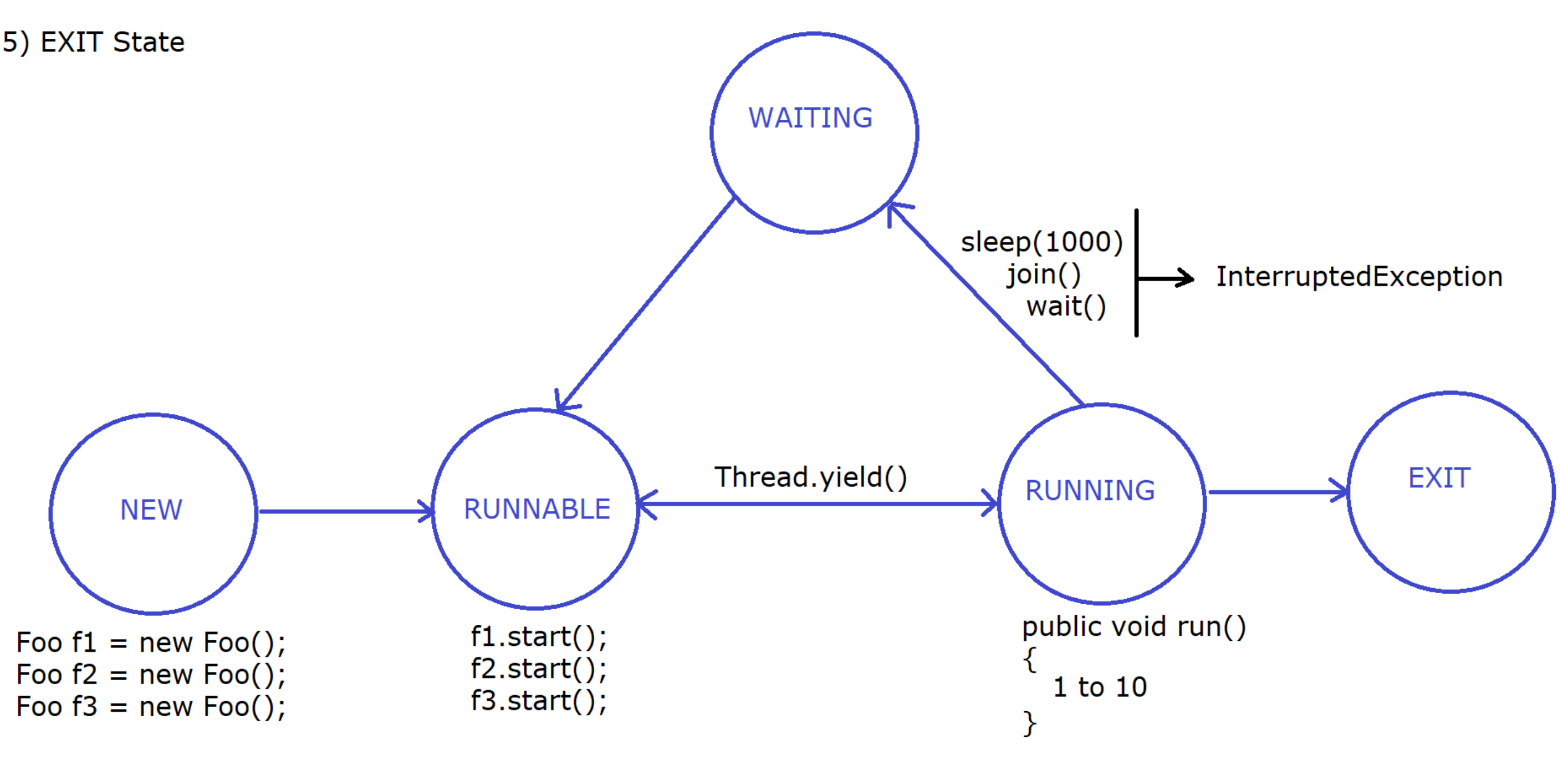
1) NEW State (Born state)

2) RUNNABLE State (Ready to run)

3) RUNNING State

4) WAITING State

5) EXIT State



New State :-

Whenever we create a thread instance(Thread Object) a thread comes to new state OR born state. New state does not mean that the Thread has started yet only the object or instance of Thread has been created. [We can set the properties]

Runnable state :-

Whenever we call start() method on thread instance, A thread moves to Runnable state i.e Ready to run state. Here Thread schedular is responsible to select/pick a particular Thread from Runnable state and sending that particular thread to Running state for execution.

Running state :-

If a thread is in Running state that means the thread is executing its own run() method in a separate runtime stack Memory.

From Running state a thread can move to waiting state either by an order of thread schedular OR developer has written some method(wait() or join() or sleep()) to put the thread into temporarly waiting state.

From Running state the Thread may also move to Runnable state directly, if developer has written Thread.yield() method explicitly.

Waiting state :-

A thread is in waiting state means it is waiting for it's time period to complete OR in some cases it is also waiting for lock (monitor) OR another thread to complete. Once the time period will be completed then it will re-enter inside the Runnable state to complete its remaining task.

Dead or Exit :

Once a thread has successfully completed its own run method in the corresponding stack then the thread will move to DEAD state. Please remember once a thread is dead we can't restart a thread in java.

IQ :- If we write Thread.sleep(1000) then exactly after 1 sec the Thread will re-start?

Ans :- No, We can't say that the Thread will directly move from waiting state to Running state.

The Thread will definetly wait for 1 sec in the waiting state and then again it will re-enter into Runnable state which is control by Thread Schedular so we can't say that the Thread will re-start just after 1 sec.

Anonymous inner class by using Thread class with reference variable :

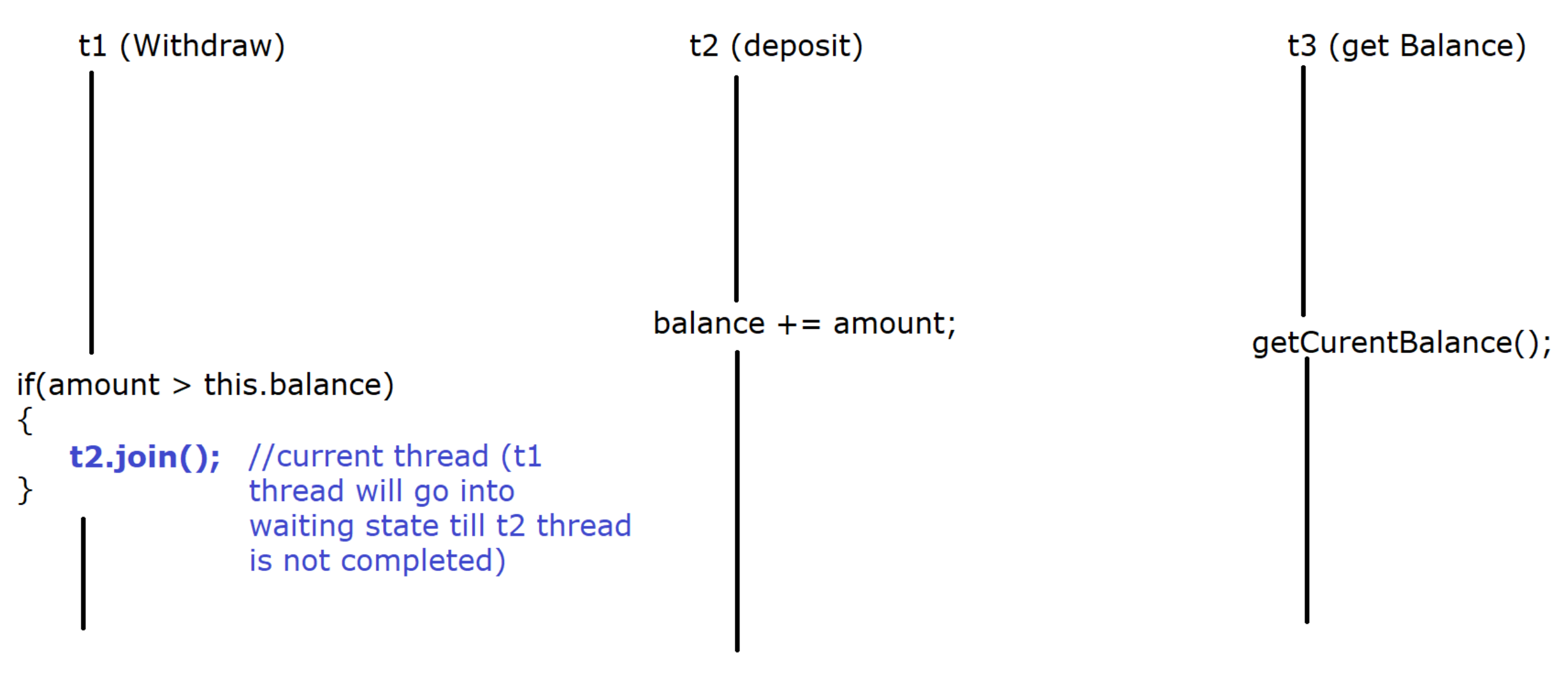
```
package com.ravi.anonymous;

public class AnonymousWithReference
{
    public static void main(String[] args)
    {
        Thread t1 = new Thread()
        {
            @Override
            public void run()
            {
                String name = Thread.currentThread().getName();
                IO.println("Running thread name is :"+name);
            }
        };

        t1.start();

    }

}
```

Anonymous inner class without reference :

```
package com.ravi.anonymous;

public class AnonymousWithoutReference {

    public static void main(String[] args)
    {
        new Thread()
        {
            @Override
            public void run()
            {
                String name = Thread.currentThread().getName();
                IO.println("Running thread name is :"+name);
            }
        }.start();

    }
}
```

---

public void join() throws InterruptedException :


t1 (Withdraw)
t2 (deposit)
t3 (get Balance)
balance += amount;
getCurentBalance();
if(amount > this.balance)
{
t2.join();
}
//current thread (t1 thread will go into waiting state till t2 thread is not completed)